

AI Personal Wardrobe & Virtual Fashion Platform

Technology Stack Reference

Document version 1.0 • June 18, 2026 • wardrobe-ai/

1. Executive Summary

This platform is a multi-service, Docker-first application for AI-powered wardrobe management, face authentication, outfit styling, and virtual fashion. It combines a JavaScript web stack with Python microservices for machine learning, PostgreSQL for relational data, and Qdrant for vector search.

2. Languages Used

Language	Where Used	Why
TypeScript	Backend (NestJS)	Strong typing, safer API contracts, NestJS ecosystem
JavaScript (JSX)	Frontend (Next.js)	React/Next convention, fast UI iteration
Python 3	AI microservices	Best ML libraries: InsightFace, PyTorch, rembg, OpenCV
SQL	Migrations & schema	PostgreSQL relational integrity and indexes
YAML / Shell	Docker & scripts	Reproducible infrastructure-as-code

3. Architecture

Browser !' Next.js Frontend (3003 dev / 3000 Docker) !' NestJS API Gateway
Qdrant (6333), Face Service (8000), Stylist Service (8001).

- Docker-first: single docker-compose.yml for six core services
- Internal network: wardrobe-network with container hostnames
- Graceful degradation: backend fallbacks when AI services are down
- Local-first: no cloud required for development

4. Frontend Stack

Core

Technology	Version	Why
Next.js	15.x	App Router, SSR, image optimization, API rewrites
React	19.x	Component UI, large ecosystem
Tailwind CSS	3.4.x	Rapid styling, design tokens, small bundles

UI & State

- Radix UI — accessible dialogs, tabs, select (WAI-ARIA primitives)
- ShadCN pattern (CVA + clsx + tailwind-merge) — customizable components
- Zustand — minimal global client state (auth, wardrobe cache)
- TanStack Query — server state, caching, optimistic mutations
- React Hook Form + Zod — performant forms with schema validation
- Lucide React + next-themes — icons and dark/light mode

Face Auth & Media

- @vladmandic/face-api + TensorFlow.js — browser face detection for camera preview
- browser-image-compression — WebP upload compression (~500KB, 1024px max)

- Next.js Image — lazy loading, responsive sizes, WebP/AVIF

5. Backend Stack (NestJS Gateway)

Library	Purpose	Why
NestJS 10	API framework	Modules, DI, guards, enterprise structure
JWT + Passport	Authentication	Stateless sessions, standard Nest pattern
bcryptjs	Password hashing	Secure credential storage
class-validator	DTO validation	Decorator-based request validation
@nestjs/swagger	API docs	OpenAPI at /api/docs
pg	PostgreSQL	Relational data, migrations
@qdrant/js-client-rest	Vector DB	Face similarity search
Jest	Testing	Unit tests for services

6. AI Services (Python / FastAPI)

Face Service — Port 8000

- InsightFace + ONNX Runtime — 512-dim ArcFace embeddings for lo
- OpenCV + Pillow + NumPy — image preprocessing
- Isolated service keeps heavy ML models out of the Node gateway

Stylist Service — Port 8001

- PyTorch + torchvision (MobileNetV2) — clothing category & 512-dim
- scikit-learn K-Means — dominant color extraction
- rembg (U-2-Net) — automatic background removal on wardrobe uplo
- FastAPI + Uvicorn + Pydantic — consistent API layer with face-servic

7. Databases & DevOps

Technology	Port	Role
PostgreSQL	5432	Users, wardrobe, outfits (ACID relational)
Qdrant	6333	Face & clothing vector similarity search
Docker Compose	—	6-service reproducible stack
Named volumes	—	Persistent data across restarts

8. Local Service URLs

Service	URL
Frontend (npm dev)	http://localhost:3003
Frontend (Docker)	http://localhost:3000
Backend API	http://localhost:3001/api
Swagger UI	http://localhost:3001/api/docs
Face AI Service	http://localhost:8000
Stylist AI Service	http://localhost:8001
PostgreSQL	localhost:5432
Qdrant	http://localhost:6333

9. Key Design Decisions

Decision	Why This Stack Won
Next.js over plain SPA	Routing, image optimization, API rewrites, SSR
NestJS over raw Express	Modules, guards, Swagger, scales with features
Python for AI	InsightFace, rembg, PyTorch — best ML ecosystem
Postgres + Qdrant	Relational integrity + local vector search
Zustand + React Query	Less boilerplate; clear client vs server state
Docker Compose (not K8s yet)	Right size for local dev and MVP

10. Feature-to-Technology Map

Feature	Technologies
Face auth	face-api.js, InsightFace, Qdrant, JWT
Wardrobe catalog	Next.js, NestJS, PostgreSQL, stylist analysis
Background removal	rembg + U-2-Net (stylist-service)
Outfit styling	NestJS outfits, stylist recommendation engine
Image optimization	browser-image-compression, Next.js Image